ChatGPT

Minecraft 本地 AI 建筑器工程方案与参考资源调研报告

结论摘要

一句话路线：**把开发基线固定在 Minecraft Java Edition 1.20.4，服务器侧用 PaperMC [1] + WorldEdit 作为落地环境，AI 只输出受约束的 JSON BuildSpec，本地脚本负责校验并导出 `.schem`，最后你手动 `//schem load` / `//paste` 导入。** 这条路线最符合“个人项目、简单、稳定、可实现”的要求，因为它避开了实时 bot、任务队列、容器编排和多服务部署，同时又保留了以后再接 mineflayer 或 `.nbt` 导出的扩展空间。[2]

```
自然语言需求
  ↓
LLM（带 block_catalog + examples + build_schema）
  ↓
build_spec.json
  ↓
本地校验器（schema / block id / bounds / 支撑关系）
  ↓
导出器
  ├── .schem   ← 首选，给 WorldEdit 导入
  └── .litematic / .nbt   ← 选配
  ↓
Paper 服务器 + WorldEdit
  ↓
你手动导入并检查结果
  ↓
可选第二阶段：mineflayer bot 自动建造
```

核心决策可以直接定下来：

- **Minecraft 版本：1.20.4。** 这是一个“参考项目、执行环境、数据层”都很好对齐的版本；MC-Bench 的简化 demo 直接固定到 1.20.4，而 WorldEdit、Litematica、mineflayer、minecraft-data 都覆盖 1.20.x。[3]
- **执行路径：A，先做 `.schem` / 蓝图导入。** 这条线最稳定，因为 WorldEdit 原生支持 schematic/clipboard 的保存与加载，而 Python 侧已有成熟库能直接写 schematic 或 litematic。[4]
- **AI 输出格式：不要让模型直接写二进制 schematic；用 JSON BuildSpec 作为模型原生输出，再由导出器生成 `.schem`。** 这与 Sponge schematic 规范、MineBench 的“先结构化输出再转换/展开”的思路更一致，也更可调试。[5]

版本选择

我建议你把**开发目标版本锁定为 1.20.4**，并把“服务器执行环境”和“客户端预览环境”分开看：服务器侧是 **Paper 1.20.4 + WorldEdit**；客户端如果你以后想要幽灵蓝图预览，再额外配 **Fabric 1.20.4 + Litematica**。WorldEdit 官方文档明确说明它同时兼容 NeoForge、Fabric、Bukkit、Spigot、Paper、Sponge，而且在服务器场景下明确推荐 Paper 胜过 Spigot；Litematica 当前也覆盖 1.20.1–1.20.6；mineflayer 支持 1.8 到

1.21.11；minecraft-data 也明确包含 1.20.4、1.20.5、1.20.6 的数据版本。把版本锁在 1.20.4，能够同时吃到这些生态的“成熟度”和“示例对齐度”。 [6]

我**不建议直接追最新版本**，不是因为最新一定不能用，而是因为“个人项目”最怕的是**工具链小版本错位**。以当前时间点为例，Paper 的下载页已经是 1.21.11，mineflayer README 也写到支持 1.21.11，但 minecraft-data README 只明确列到 1.21.10，`minecraft-assets` 仓库则列到 1.21.8；这说明在 1.21.x 这条线上，核心执行层、协议层、数据层、资源层的最新小版本并不是完全同步的。对大团队这不算大问题，但对单人本地项目，它很容易演化成“为什么这个 block/state/asset 在 A 库可用，在 B 库还没跟上”的时间黑洞。这里我的判断是一个工程性推断，但它直接建立在这些仓库当前宣称的支持版本差异之上。 [7]

我也**不建议退回太老的版本**，尤其是 1.12.x 及更早。WorldEdit 文档直接说明，在 WorldEdit 7 之前，对应的是旧 `.schematic` 格式；从 Minecraft 1.13 开始，Mojang 的 blockstate 体系变化后，旧格式已经不适合新 block format，所以 WorldEdit 才转向了 Sponge schematic 的 `.schem`。今天做一个新项目，如果你从老版本起步，等于一开始就把自己绑回旧格式兼容层。 [8]

如果你确实想在客户端做“看着蓝图一层层搭”的体验，**Fabric 比 Forge 更自然**。Litematica 官方描述写得很清楚：它从 1.14+ 主要面向轻量 loader，现代版本核心是 Fabric 路线；Forge 官方支持止于 1.12.2，之后更多依赖第三方移植。这意味着你的 MVP 不应该围着 Forge 组织。 [9]

执行路径选择

下面这三条路径里，我建议你按 **A → B → C** 的顺序推进，而不是一开始就上 bot 自动建造。

路径	适合你的原因	主要问题	结论
A. `.schem` / 蓝图导入	WorldEdit 原生支持 clipboard 与 schematic 的保存、加载和粘贴；mcschematic 与 Litemapy 都能通过代码创建可导入结构文件，因此“AI 生成结构化方案 → 本地转换器 → 玩家导入”是天然顺滑的。 [4]	需要你自己维护一个导出器；如果你选纯香草 `.nbt` 结构文件，Java 结构方块单次保存/加载尺寸上限是 48×48×48，更适合小型建筑而不是随意放大的大体量作品。 [10]	**最推荐。** 其中最终交换格式优先选 `.schem`，不是 `.nbt`。
B. WorldEdit 命令执行	WorldEdit 自身就提供 `//set`、复杂 pattern、表达式、clipboard 等能力，所以如果 AI 只是在做规则几何、修补、替换、旋转，其实直接产生命令是能工作的。 [11]	命令是高度**状态化**的：要先有 selection、再 copy/paste、还要处理相对位置、mask、权限和命令顺序。对 LLM 来说，这通常比生成一个可验证的 JSON 结构更脆弱。这个“更脆弱”的判断是工程推断，但它直接来自 WorldEdit 这套命令对 selection、clipboard、relative position 的依赖。 [12]	**适合做二号通道。** 用来修形、局部操作、后处理，而不是主路径。
C. bot 自动建造	mineflayer 已经支持 building、digging、inventory 等；MC-Bench 的 orchestrator 也证明了 mineflayer 可以成为“模型输出 → Minecraft 执行”的核心执行器。 [13]	这是三条路径里移动部件最多的一条：需要 live server、bot 登录、节流、路径/权限处理、失败恢复；MC-Bench 为了这件事引入了 Redis、Docker、server manager、batch processing 和日志体系，复杂度明显高于个人 MVP 需要。 [14]	**放到第二阶段。** 等你的 schema、导出器、方块知识库稳定以后再接。

明确推荐：**你的第一版只做 A。** 更具体地说，是做 **“AI 输出 JSON BuildSpec → 本地导出 `.schem` → 你手动导入”**。如果某些特殊模型或工具更擅长生成逐步建造脚本，你也可以短期保留 B 作为辅助通道；但 C 不应该是第一天的主线。 [15]

参考资源清单

这一节同时承担你要求的“执行桥、数据层、结构格式、工具链”四组资源清单。判断标准只有一条：**是不是适合单人本地项目的 MVP。**

执行桥

名称	作用	是否推荐
WorldEdit	你的**首要落地终点**。它原生支持 copy/paste、`//schem save/load`、clipboard 与可扩展格式，是 `.schem` 导入链路最稳的最后一公里。 [16]	**推荐**
mcschematic	Python 侧最直接的 `.schem` 生成库之一；支持按坐标 `setBlock`、保存为指定 Minecraft 版本兼容的 schematic，非常适合你把 AI 的 JSON 输出编译成导入文件。 [17]	**强烈推荐**
Litemapy	如果你想额外支持 `.litematic` 和客户端蓝图预览，它很合适；文档里有 region、坐标系统、load/save 示例。 [18]	**选配推荐**
mineflayer	bot 自动建造的核心桥接层；支持 building/digging，版本覆盖很广。若以后做“AI 真正在世界里搭方块”，基本绕不开它。 [19]	**第二阶段推荐**
MC-Bench simple-minecraft-server-builder	一个很适合“借骨架、不照抄全栈”的参考 repo：它提供本地 1.20.4 服务器、prompt 模板、`safeSetBlock()` / `safeFill()` 帮助函数、脚本模板，以及 `.nbt` 导出工作流。 [20]	**推荐参考其模板与 helper 思路**

数据层

名称	作用	是否推荐
`minecraft-data`	提供 blocks、items、materials、recipes、block collision shapes、versions 等，是你的**canonical registry**。它本身也是 mineflayer 等项目的数据底座。 [21]	**强烈推荐**
`prismarine-block`	给你补足单个 block 的工程字段，比如 `name`、`displayName`、`shapes`、`hardness`、`boundingBox`、`transparent`、`diggable`、`material`，很适合做“AI 可消费”的 block catalog。 [22]	**强烈推荐**
`minecraft-assets` / `mc-assets`	提供原版资源与辅助 JSON，用来拿贴图、做颜色聚类、生成缩略图或离线材质参考很合适。`mc-assets` 还是 MC-Bench 团队对上游资源仓库的 Python 包装。 [23]	**推荐**
`python-minecraft-data` / `minebase`	如果你的实现主语言是 Python，这两者能让你不必自己桥接 JS 数据仓库。前者是老牌 Python wrapper，后者是较新的 Python wrapper around PrismarineJS/minecraft-data。 [24]	**按语言选配**

结构格式

名称	作用	是否推荐
Sponge `.schem`	现代 schematic 交换格式；规范目标就是比旧 MCEdit schematic 更适合新版本、更有前向兼容性，并支持 block states、Entities、BlockEntities。WorldEdit 也把它作为 1.13+ 之后的新主格式。 [25]	**最终导出格式首选**
`.litematic`	适合客户端蓝图预览；支持 region、metadata 和更明确的坐标系统，但它不是服务器侧最通用的导入终点。 [18]	**预览用途推荐**
Java 结构文件 `.nbt`	纯香草路线可用；结构方块可以把结构保存到世界目录下的 `generated/<namespace>/structures`，simple builder 也直接输出到这个位置。但 Java 结构方块尺寸限制明显，更适合小体量。 [10]	**仅作小型/纯香草备选**
MineBench Build JSON	不是 Minecraft 原生格式，但非常适合作为**LLM 原生输出格式参考**：它把最终构建 spec 表示为 `version/boxes/lines/blocks`，先可验证，再可展开，再可转换。 [26]	**内部 AI 输出格式强烈推荐参考**

工具链

名称	作用	是否推荐
Paper	WorldEdit 文档对服务器侧明确推荐 Paper；它是你本地执行与导入环境的最佳默认选择。 [27]	**强烈推荐**
FastAsyncWorldEdit	如果你后面 paste 很大的结构，FAWE 是很自然的升级项；它主打性能与内存改进，兼容 1.20.x / 1.21.x。 [28]	**大结构时推荐**
Amulet	它既是编辑器，也是可跑 Python operation 的世界编辑工具；如果你以后想直接改世界存档、做检查器或批处理，Amulet 有价值，但它比 `.schem` 路线更重。 [29]	**选配**
GitHub Copilot 自定义说明与 prompt files	用来给 Copilot 固定项目上下文、构建/测试命令、schema 约束与常用提示，非常适合你这种“结构受约束的生成项目”。官方文档支持 `.github/copilot-instructions.md`、`.github/instructions/*.instructions.md`、`AGENTS.md` 和 `.github/prompts/*.prompt.md`。 [30]	**强烈推荐**

方块数据与建筑语义层

如果只问“有没有现成库给我 block 数据”，答案是**有，而且很成熟**。`minecraft-data` 提供 blocks、materials、recipes、block collision shapes 等版本化数据；`node-minecraft-data` 文档还明确给出

`blocksByName`、`blocksByStateId`、`blockCollisionShapes` 这些访问面。与此同时，`prismarine-block` 进一步补充了 AI 真正常用的字段：`displayName`、`shapes`、`hardness`、`boundingBox`、`transparent`、`diggable` 和 `material`。[31]

但如果你问“这些库有没有把**颜色、材质审美、建筑风格**直接喂给 AI”，答案基本是**没有**。`minecraft-data` 里的 `materials` 更偏向工具速度/挖掘体系；`prismarine-block` 里的 `material` 也是 `rock / wood / plant / wool` 这种偏物理与采掘的机械分类，而不是“暖色木材”“宫殿风”“中世纪屋顶”这种建筑语义。至于颜色与纹理，现成仓库更多是给你**贴图资产**和相关 JSON，而不是现成的审美标签。所以，AI 不应该全靠模型记忆去猜；正确做法是：**用现成 registry + 贴图资源做底座，再自己补一层轻量语义标签。**[32]

我建议你的 `block_catalog.json` 不要追求“全方块百科全书”，而是做成一个**面向建筑生成的轻量 catalog**，只保留常用建筑块、常见变体、结构性约束和审美标签。底层 canonical id 来自 `minecraft-data`；物理与可交互属性来自 `prismarine-block`；颜色/明度/风格标签则来自你对原版贴图的离线归类或人工补充。这样做的好处是，AI 看到的是一个“足够小、足够稳、不会胡用奇怪方块”的材料表，而不是把整个 Minecraft 数据宇宙直接塞进 prompt。[33]

一个适合你项目的字段结构可以长这样：

```
{
 "mc_version": "1.20.4",
 "blocks": [
  {
   "id": "minecraft:oak_planks",
   "display_name": "Oak Planks",
   "category": "solid_block",
   "visual_material": "wood",
   "color_family": "brown",
   "lightness": "medium",
   "texture_pattern": "planks",
   "style_tags": ["medieval", "rustic", "starter_house"],
   "variants": {
    "slab": "minecraft:oak_slab",
    "stairs": "minecraft:oak_stairs",
    "fence": "minecraft:oak_fence",
    "door": "minecraft:oak_door"
   },
   "physics": {
    "bounding_box": "block",
    "transparent": false,
    "diggable": true,
    "gravity": false
   },
   "placement": {
    "orientation_props": [],
    "waterloggable": false,
    "needs_support": false,
    "tile_entity": false
   },
   "usage": {
    "roof_ok": true,
```

```
      "wall_ok": true,
      "foundation_ok": false,
      "detail_ok": true
    }
  }
 ]
}
```

这个 schema 里，`id / display_name / physics / placement` 这些是“硬约束”；`visual_material / color_family / texture_pattern / style_tags / usage` 是“AI 建筑语义层”。前者帮助你**校验**，后者帮助你**选材**。从工程角度看，这是比“把所有方块说明文本直接喂给模型”更稳定的做法。支撑这个拆法的底座能力，正好对应 `minecraft-data`、`prismarine-block` 和资源仓库各自擅长的部分。 [33]

推荐的 AI 输出格式

关于“AI 输出 Minecraft 建筑时，常见格式有哪些”，你在工程上其实应该把它拆成两层看：

- **模型原生输出层**：JSON BuildSpec
- **Minecraft 交换/导入层**：`.schem`
- **可选预览层**：`.litematic` 或 `.nbt`

我不建议让模型直接生成 `.schem` 二进制文件。原因很简单：Sponge schematic 规范自己就说明，它本质上是一个**存储规范**，不是给 consumer app 当对象模型直接操作的 LLM 友好格式；而现成的“AI 生成建筑”实践里，无论是 MineBench 的 `version/boxes/lines/blocks`，还是第三方的 Claude Minecraft builder，都是先让模型产出结构化 JSON，再做验证、展开、转换，最后才导出 `.schem`。 [5]

因此，我给你的明确推荐是：**让 AI 输出“JSON 里的 DSL”，而不是“纯坐标 JSON”也不是“直接 schematic”**。 也就是，用 JSON 包一组原语操作，例如 `block`、`box`、`line`、`fill_hollow`、`stairs_strip`。这样做的好处是：比纯坐标紧凑，比自由文本稳定，比二进制格式可调试，而且和 MineBench 的最小原语设计高度一致。 [26]

一个适合你的 BuildSpec 示例可以是这样：

```
{
 "version": "1.0",
 "mc_version": "1.20.4",
 "name": "medieval_cottage",
 "size": { "x": 9, "y": 8, "z": 7 },
 "palette": {
  "foundation": "minecraft:cobblestone",
  "wall": "minecraft:oak_planks",
  "frame": "minecraft:spruce_log[axis=y]",
  "roof": "minecraft:spruce_stairs[facing=east,half=bottom,shape=straight]",
  "glass": "minecraft:glass_pane"
 },
 "ops": [
  { "op": "box", "from": [0, 0, 0], "to": [8, 0, 6], "block": "foundation" },
  { "op": "box_hollow", "from": [0, 1, 0], "to": [8, 4, 6], "block": "wall" },
  { "op": "line", "from": [0, 1, 0], "to": [0, 4, 0], "block": "frame" },
  { "op": "line", "from": [8, 1, 0], "to": [8, 4, 0], "block": "frame" },
```

```
  { "op": "set", "at": [4, 2, 0], "block": "glass" }
 ]
}
```

本地编译器要做的事很确定：先根据 `build_schema.json` 做 JSON Schema 校验；再检查 palette 里的 block id 是否存在于 `block_catalog.json`；然后把 `ops` 展开为离散方块；最后导出为 `.schem`。如果以后你想兼容更长的建筑，可以再加入 `repeat`、`mirror`、`rotate90` 之类的高阶原语，但第一版不要太贪。 [34]

MC-Bench 可借鉴部分拆解

MC-Bench 的价值不在于“你应该把它整套搬过来”，而在于它已经把“模型输出、执行、存档、评测”这些角色拆得很清楚。它的 orchestrator README 明示了 `build_service.py`、`server_manager.py`、`mineflayer.py`、`test.py` 的分工：任务进入队列，服务拉任务，创建 Minecraft server，调用 mineflayer 执行，保存结果，再清理环境。与此同时，后端仓库本身又有 Docker、OAuth、数据库迁移、前后端联动等完整应用栈。这正说明：**MC-Bench 是很好的“模块边界参考”，但不是很好的“个人项目骨架”。** [14]

模块	是否推荐借鉴	原因
`function_definition + metadata` 这种“任务协议”	**推荐精简借鉴**	orchestrator 的 job object 已经把“构建逻辑”和“构建元数据”分开了；你完全可以保留这种思想，但把执行载体改成 JSON BuildSpec，而不是远程函数定义。 [35]
`build_service.py` 这种“协调层”	**推荐借鉴思路，不推荐照搬实现**	你需要一个本地 coordinator，但它只需要负责“读 JSON → 校验 → 导出 `.schem`”，不需要 Redis worker。 [36]
`server_manager.py` 容器化临时服务器	**不推荐**	MC-Bench 这么做是为了并发评测和隔离环境；个人项目直接本地固定一个 Paper 世界更简单。 [36]
`mineflayer.py` 的命令队列、放置节流、坐标跟踪	**第二阶段推荐借鉴**	如果你未来真的做 bot，这部分很值钱；但在 `.schem` MVP 阶段，它会把你拉进不必要的实时执行复杂度。 [37]
Redis 队列、batch size、并发批处理	**不推荐**	这是 benchmark / service 化场景的需求，不是本地单人玩法的需求。 [36]
后端全栈：Docker Compose、数据库迁移、GitHub OAuth、前端登录	**不推荐**	这些在 MC-Bench 后端里是合理的产品化能力，但与你的本地 builder 目标无关。 [38]
`simple-minecraft-server-builder` 的模板、prompt、helper 函数	**强烈推荐**	这个仓库恰好提供了你需要的最小骨架：本地 server、prompt 模板、`safeSetBlock()` / `safeFill()`、脚本模板、导出文件路径。它比完整后端更贴近你的实际需求。 [20]
`.nbt` 导出到世界目录的流程	**部分推荐**	如果你要保留一个“纯香草导入”后门，这个流程很有参考价值；但主线仍应是 `.schem`。 [39]

所以，MC-Bench 对你最有价值的不是“部署它”，而是这三个工程结论：**角色要分层、执行接口要收窄、输出工件要可保存可复现。** 你只借这三点，就已经足够。 [40]

Copilot 项目准备材料映射

这一部分直接对应你要喂给 Copilot 的“材料清单”。按 GitHub [41] 官方文档，`.github/copilot-instructions.md` 用来放 repo-wide 的长期指导；`.github/instructions/*.instructions.md` 可以给特定路径加规则；`AGENTS.md` 用来给 agent 模式提供上下文；而 prompt files 则放在 `.github/prompts/*.prompt.md`。同时，官方还给出了 README、onboarding plan、API 文档等 prompt file 范式，这些都很适合你这种强 schema、强约束的生成项目。[42]

材料文件	参考来源	建议内容
`PROJECT_BRIEF.md`	MC-Bench simple builder 的 README；WorldEdit 的安装与 clipboard 文档。[43]	用一页写清：项目目标、**锁定版本 1.20.4**、首选执行路径是 `.schem`、为什么不做 bot first、文件输出约定、允许的 block catalog 范围、非目标（不做多用户竞技、不做分布式任务系统）。
`.github/copilot-instructions.md`	GitHub 官方自定义说明文档。[44]	写 repo-wide 规则：技术栈、运行命令、测试命令、目录职责、版本钉死、**永远先改 schema 再改 prompt**、**不要让模型直接输出二进制 schematic**、必须校验 block id 和 bounds、所有导出要可复现。
`docs/architecture.md`	MC-Bench orchestrator 的分层思路；WorldEdit 的执行终点文档。[45]	画清组件边界：LLM、block catalog、schema validator、exporter、import target。特别写明“内部表示”和“外部交换格式”是两层：内部 JSON，外部 `.schem`。
`schema/build_schema.json`	MineBench 的 `version/boxes/lines/blocks` 思路；Claude Minecraft builder 的 JSON 结构。[46]	定义 BuildSpec：`version`、`mc_version`、`name`、`palette`、`ops`、坐标范围、允许原语、禁止未知字段。把“模型可输出什么”写成机器可校验规则。
`data/block_catalog.json`	`minecraft-data`、`prismarine-block`、`minecraft-assets` / `mc-assets`。[32]	收录常用建筑块及其语义标签：`id`、变体、透明度、碰撞、风格标签、颜色族、材质、用途。它既是 prompt 上下文，也是校验白名单。
`examples/`	GitHub prompt files 库；MC-Bench simple builder；Claude Minecraft builder examples。[47]	放 3 类样例：自然语言请求、对应 BuildSpec JSON、导出的 `.schem`/渲染图。至少覆盖“小屋、塔楼、拱门、围墙、屋顶”几类。最好再放一个“坏例子”告诉 Copilot 什么输出会被 validator 拒绝。

如果只保留最小交付，我会保留以上六项；但如果你想让 Copilot 真正“少走弯路”，我还会顺手加三样**不在你清单里、但很值钱**的文件：`AGENTS.md`、`.github/prompts/plan-build.prompt.md`、`.github/prompts/review-build-json.prompt.md`。前者给 agent 模式固定项目规则，后两者分别让 Copilot 更稳定地做“规划建筑 spec”和“审查 spec 是否违背 schema”。这些做法都直接符合 GitHub 官方对 custom instructions、path-specific instructions、agent instructions 和 prompt files 的组织方式。[30]

---

1 13 19 GitHub - PrismarineJS/mineflayer: Create Minecraft bots with a powerful, stable, and high level JavaScript API. • GitHub
https://github.com/prismarinejs/mineflayer

2 3 20 39 43 https://github.com/mc-bench/simple-minecraft-server-builder
https://github.com/mc-bench/simple-minecraft-server-builder

4 8 12 16 Clipboard - WorldEdit 7.4 documentation
https://worldedit.enginehub.org/en/latest/usage/clipboard/

5 15 25 https://github.com/SpongePowered/Schematic-Specification/blob/master/README.md
https://github.com/SpongePowered/Schematic-Specification/blob/master/README.md

6 https://github.com/enginehub/worldedit
https://github.com/enginehub/worldedit

7 https://papermc.io/downloads/paper
https://papermc.io/downloads/paper

9 Litematica - Minecraft Mod
https://modrinth.com/mod/litematica?utm_source=chatgpt.com

10 https://minecraft.fandom.com/wiki/Structure_Block
https://minecraft.fandom.com/wiki/Structure_Block

11 Region Operations - WorldEdit 7.4 documentation
https://worldedit.enginehub.org/en/latest/usage/regions/regionops/?utm_source=chatgpt.com

14 35 36 40 45 https://raw.githubusercontent.com/mc-bench/orchestrator/main/README.md
https://raw.githubusercontent.com/mc-bench/orchestrator/main/README.md

17 https://pypi.org/project/mcschematic/
https://pypi.org/project/mcschematic/

18 https://litemapy.readthedocs.io/
https://litemapy.readthedocs.io/

21 31 32 33 https://github.com/prismarinejs/minecraft-data
https://github.com/prismarinejs/minecraft-data

22 https://github.com/PrismarineJS/prismarine-block
https://github.com/PrismarineJS/prismarine-block

23 https://github.com/PrismarineJS/minecraft-assets
https://github.com/PrismarineJS/minecraft-assets

24 https://github.com/SpockBotMC/python-minecraft-data
https://github.com/SpockBotMC/python-minecraft-data

26 https://github.com/Ammaar-Alam/minebench
https://github.com/Ammaar-Alam/minebench

27 https://worldedit.enginehub.org/en/latest/install/
https://worldedit.enginehub.org/en/latest/install/

28 FastAsyncWorldEdit - Minecraft Plugin
https://modrinth.com/plugin/fastasyncworldedit/versions

29 https://www.amuletmc.com/python
https://www.amuletmc.com/python

30 42 44 Adding repository custom instructions for GitHub Copilot - GitHub Docs
https://docs.github.com/copilot/customizing-copilot/adding-custom-instructions-for-github-copilot

34 41 46 https://raw.githubusercontent.com/Ammaar-Alam/minebench/master/docs/voxel-exec-raw-output.md
https://raw.githubusercontent.com/Ammaar-Alam/minebench/master/docs/voxel-exec-raw-output.md

37 GitHub - mc-bench/orchestrator: Worker to orchestrate and manage running an arbitrary number of LLM-generated builds concurrently using containerized Minecraft Servers. · GitHub
https://github.com/mc-bench/orchestrator

38 https://github.com/mc-bench/mc-bench-backend/blob/main/docs/getting-started/docker-guide.md
https://github.com/mc-bench/mc-bench-backend/blob/main/docs/getting-started/docker-guide.md

47 Prompt files - GitHub Docs
https://docs.github.com/en/copilot/tutorials/customization-library/prompt-files